Setup Wizard Guide

A First-Time User's Path from Delphi Project to Offline Translation

Version 1.0

Last changed: August 12, 2026

Windows • Delphi VCL and FireMonkey • Win32 and Win64

Table of Contents

1. Purpose and Expected Result

The Setup Wizard is the recommended way to localize a Delphi application for the first time. It presents one decision at a time, verifies prerequisites before allowing the next step, and records the files and commands needed to finish the integration.

At completion, the developer has a saved development catalog, compact JSON runtime language packs, a component integration kit, a required safety backup, and automatic build deployment. Pascal, DFM, FMX, and DPR files remain untouched. The Wizard adds one clearly marked, backed-up block to the DPROJ for the ComponentSource Search Path and post-build language-pack deployment.

The Wizard cannot perform Delphi's design-time work on the developer's behalf. RAD Studio must install the design package through its normal Install Packages dialog, and the developer must place and configure the language manager and visible selector on the target application's primary form. These visible IDE changes are deliberate safeguards.

> **End-user applications remain offline.** Google Cloud Translation or DeepL is contacted only by the Translation Studio on the developer's computer. The finished application reads local JSON files and requires no Internet connection, provider account, or API key.

2. Prepare Before Starting

Requirement	What to prepare	Why it matters
Target project	A .dproj file is preferred; .dpr is accepted.	The Wizard identifies VCL or FMX, supported Windows platforms, and form resources.
Safe working copy	Use a clearly named test copy and confirm it builds before localization.	The original application remains available even if the evaluation is abandoned.
Backup method	Git is optional. A verified folder copy, archive, or normal backup system is sufficient.	The Wizard does not assume every Delphi developer uses Git.
Translation provider	A working Google Cloud Translation Basic v2 or DeepL API key.	Automatic translation cannot run without provider authentication.

Requirement	What to prepare	Why it matters
RAD Studio	The current release is verified with RAD Studio 13 Florence.	RAD Studio performs the approved design-package and component-placement steps.
Healthy target build	Build at least one intended Windows configuration successfully.	A localization test should not begin with unrelated compiler or form-streaming failures.

1. Close the target application if it is running.
2. Create the pristine or otherwise protected copy, then create a separate test copy for the Wizard.
3. Open the test copy in RAD Studio, build it, run it briefly, then close RAD Studio or close the project before starting the Wizard.
4. Have the provider API key available. Do not paste it into source code, JSON files, email, screenshots, or issue reports.

3. Start and Navigate the Setup Wizard

1. Run DelphiAppTranslationStudio.exe from the selected Win32 or Win64 Debug or Release folder.
2. On the Project page, select Start Setup Wizard. The separate Open Project button begins the advanced manual workflow and is not required for this guide.
3. Read the Welcome page, especially the orange safety notice, then select Next.

Navigation control	Behavior
Next	Validates the current step and moves forward only when required information is present.
Back	Returns to the previous step before final processing.
Left step rail	Selects a step already reached. Future steps remain unavailable, preventing required work from being skipped.
Cancel	Closes the Wizard before final processing. The target project remains unchanged; a deliberately saved provider credential may remain in Windows Credential Manager.

Navigation control	Behavior
Begin Final Processing	Starts the controlled translation/export sequence. Back, Cancel, the left rail, and window closing are disabled until the sequence completes or stops safely.
Finish	Becomes available only after final processing succeeds.

Cancel boundary. Cancel is safe through the Review and Authorize page, before Begin Final Processing is selected. During final processing the Wizard cannot be closed. If a failure occurs after the controlled DPROJ update, the transaction restores the prior project configuration automatically. Pascal and form source are not rewritten.

4. Complete the Nine Wizard Steps

4.1 Step 1 - Welcome

The Welcome page summarizes the complete job and the safety boundary. No project has been selected and no target file is being written at this point.

1. Read the safety notice.
2. Select Next.

Expected result. The Delphi Project step opens and Step 1 remains selectable in the left rail.

4.2 Step 2 - Delphi Project

This step identifies the application that owns the text. Browse does not compile, execute, or modify the project. A .dproj file usually supplies the best framework and platform metadata; use .dpr only when no .dproj is available.

1. Select Browse.
2. Navigate to the test copy and select its .dproj file.
3. Confirm the displayed project name, VCL or FireMonkey framework, Windows targets, and form-resource count.
4. Confirm the detected Application ID. It is the project filename without .dproj; for example, MyApplication.dproj produces MyApplication. Use Copy ID if the value will be needed in RAD Studio.
5. If the project is wrong, select Browse again before continuing.
6. Select Next.

Expected result. The footer reports that the project was identified and no target file was changed. The exact

Application ID is displayed explicitly rather than inferred from the folder name.

4.3 Step 3 - Deployment Destinations

The Wizard detects normal Win32 and Win64 build-output folders automatically. This page is for additional installed, portable, network, or USB folders that contain—or will contain—the application executable. Entries are staged until authorized final processing, so Cancel still leaves the project and Studio-owned deployment settings unchanged.

1. Leave the list empty when the application will run only from normal Delphi build-output folders.
2. For a separate application copy, select Add Application Folder and choose the full folder, including its drive letter. Select the folder containing the executable, not the Localization or Languages subfolder.
3. Repeat Add Application Folder for every installed, portable, network, or USB destination that should receive language packs automatically.
4. Use Remove Selected to remove an obsolete or incorrect destination before final processing.
5. Read the summary, then select Next.

Expected result. Final processing deploys immediately to every configured destination that is currently available. The choices are saved as Studio-owned JSON and included in the component kit. Future Delphi builds retry them automatically; an unplugged or unavailable drive is skipped with a warning and does not fail the build.

4.4 Step 4 - Languages

Source language means the language currently written in the forms and source strings. Target language means the language the application user will select. The Wizard uses the locale code to name the JSON pack and to obtain date, time, number, currency, and text-direction defaults.

1. Leave Source language as English (United States) [en-US] when the application was authored in U.S. English.
2. Choose the target language from the list.
3. Confirm the native language name. This is the human-readable name displayed to the application's users.
4. Read the locale summary, then select Next.

Expected result. A locale such as es-ES is selected. Choosing a different target language invalidates downstream scan/catalog state so the wrong language cannot be exported accidentally.

4.5 Step 5 - Translation Service

The Wizard supports Google Cloud Translation Basic v2 and DeepL. A remembered key is stored as a Windows Generic Credential for the current Windows user. A session-only key is held in memory until the Studio closes. Keys are never written into catalogs, runtime packs, component kits, or target source files.

1. Select Google Cloud Translation or DeepL. For DeepL, also select API Free or API Pro to match the account.
2. If a key is already stored for this provider, the status reports that it is available. Otherwise paste a new key into the masked API key field.
3. Leave Remember securely on this computer selected for Windows Credential Manager storage, or clear it for this Studio session only.
4. Select Save / Replace Key when a new key was entered.
5. Select Test Connection and wait for Connection test passed. Do not continue after an authentication, billing, quota, restriction, or network failure.
6. Select Next.

> **Expected result.** The provider connection succeeds using one small test translation. No target application text has been sent yet.

4.6 Step 6 - Scan Project

Scanning reads text DFM/FMX resources and Delphi resourcestring declarations. It creates stable keys such as form.component.property and unit.symbol. If a development catalog already exists for the selected project and language, matching translations are preserved; only new, changed, or unresolved entries require provider work.

The scan count is not the same as the translation count. Dynamic values, identifiers, excluded entries, obsolete entries, and already complete translations may be scanned but deliberately withheld from automatic translation.

1. Select Scan Project.
2. Review the summary for total, new, changed, unchanged, and obsolete entries.
3. Scroll through representative rows and confirm that keys and source text belong to the selected project.
4. If the count is zero, stop and verify that the forms are text resources and that the correct project was selected.
5. Select Next.

> **Expected result.** The footer reports that the development catalog is ready. The scan itself remains read-only with

respect to the target project.

4.7 Step 7 - Delphi Component

This page explains the remaining manual RAD Studio phase that cannot safely be performed by the Wizard. The design-time BPL makes the manager and selector available on the DAT Localization Tool Palette page. Installing it is normally a one-time operation for the current RAD Studio installation; placing and arranging components remains designer-owned work.

1. Read every project-specific instruction shown on the page.
2. Optionally select Show Design BPL now to confirm that the exact verified VCL or FMX Win32 Release design package exists. File Explorer selects the file; the Wizard does not install it.
3. Select I understand this manual RAD Studio step.
4. Select Next.

Expected result. The Wizard records that the manual step is understood; no BPL registry entry and no target-form component has been created yet.

4.8 Step 8 - Review and Authorize

This is the final cancellation point. The summary identifies the exact project, framework, target language, provider, scan count, unresolved translation count, component integration method, and backup choice. Read the project path character by character, especially when pristine and test folders have similar names.

1. Confirm the project path and application name.
2. Confirm the target language and provider.
3. Compare Scanned entries with Unresolved entries to translate. A lower unresolved count is normal when entries are protected, excluded, obsolete, or already translated.
4. Close the target project in RAD Studio, then select Required: the target project is closed in RAD Studio. This prevents the IDE from overwriting the controlled DPROJ update with an older in-memory copy.
5. Confirm that the required timestamped ZIP safety backup is selected. It cannot be disabled because final processing adds controlled properties inside Delphi's native Base compiler group.
6. Select I reviewed these choices and authorize final processing.
7. Select Begin Final Processing only when every item is correct.

Expected result. The Wizard moves to Process and Finish, disables cancellation and navigation, and begins the logged operation sequence.

4.9 Step 9 - Process and Finish

Final processing runs in a protected single pass: required ZIP backup, automatic translation of eligible unresolved entries, initial catalog save, automatic Localization Review, application of saved terminology/layout decisions, final validation, runtime-pack export, component-kit generation, deployment to existing output folders and every available configured application destination, backed-up transactional DPROJ configuration, and completion-report creation.

Machine translations are recorded with Google or DeepL provenance. Existing Reviewed, Approved, Excluded, and Obsolete entries are not overwritten. Blocking validation errors stop the sequence; warnings are recorded but do not necessarily prevent export.

1. Watch the timestamped progress log. Do not terminate the Studio or Windows while processing is active.
2. When processing completes, select the blue component-kit path or Open Kit Folder.
3. Read the repeat/troubleshooting explanation. Normally, neither deployment button must be clicked: final processing has already deployed detected outputs and configured application destinations.
4. Build each target configuration you actually intend to ship or test. A Delphi build produces one selected platform/configuration at a time unless you use Project > Build All Projects with the required configurations enabled. The Wizard's post-build step automatically deploys the current JSON packs to that build's executable Localization\Languages folder.
5. Use Redeploy Build Outputs only to repeat deployment after an unusual manual change. Use Deploy New App Folder only for a new or temporary folder that was not entered on Step 3. The Wizard verifies the EXE before copying anything.
6. Use Copy Fallback Commands only for troubleshooting or a deliberately manual deployment.
7. Select Finish after reviewing the completion results. The Wizard closes and returns to the Studio; continue with the manual RAD Studio steps in Chapter 5.

> **Expected result.** The progress log ends successfully, the component-kit folder exists, Wizard-Completion-Report.txt records the exact Application ID, Search Path, backups, and deployments, and the DPROJ contains one marked Wizard block.

5. Complete the Manual RAD Studio Step

The Wizard creates files but deliberately does not alter RAD Studio's package registry or place components on forms. Complete these steps after final processing. If DAT Localization is already visible in the Tool Palette, do not reinstall the package; proceed to component placement.

5.1 Install the Design Package

1. Start RAD Studio without opening the target form. If the target project opens automatically, close the project before changing installed packages.
2. In the Wizard select Show Design BPL, or browse to the exact path recorded in the completion report.
3. In RAD Studio choose Component > Install Packages.
4. Select Add, choose DATLanguageManagerFMXDesign.bpl for an FMX target or DATLanguageManagerVCLDesign.bpl for a VCL target, and select Open.
5. Confirm that the DAT Language Manager design-time package appears in the package list and is checked, then select OK.
6. Open any matching VCL or FMX form and confirm that the DAT Localization Tool Palette page contains the language manager and language combo-box components.

> **Use the correct Delphi command.** Do not use Component > Install Component, do not select a .dpk, and do not install a BPL from a generated per-project kit. Install the stable Win32 Release design BPL selected by the Wizard.

5.2 Place and Configure the Components

1. Open the target test project and its primary form in the Form Designer.
2. Place one TDATFMXLanguageManager for an FMX project or one TDATVCLLanguageManager for a VCL project. One manager supervises the application; it is not placed on all forms.
3. Set ApplicationId to the exact value shown in Detected Application ID. For example, MyApplication.dproj produces MyApplication regardless of the containing folder name.
4. Leave LanguagesFolder as Localization\Languages unless the application has an intentional custom deployment layout.
5. Set SourceLanguage to en-US when English (United States) is the source.
6. Place the matching TDAT language combo box. In Object Inspector, set its LanguageManager property to the manager component; do not leave this property blank. Position it where users can select a language. A connected Language menu may replace the combo box, but a visible language choice is required.
7. Save the form and project. Delphi makes the normal visible form-resource, Pascal-field, and uses-clause changes. The Wizard has already configured the DPROJ Search Path and post-build deployment.

5.3 Verify the Automatic Search Path

The Wizard adds the generated kit's ComponentSource directory under the DPROJ Base configuration. Base inheritance makes the same path available to Debug and Release on both Win32 and Win64. No four-way manual entry is normally required.

To verify or repair the setting manually in RAD Studio, open Project > Options. Select Building > Delphi Compiler. At the top of the page select All configurations and All platforms, then locate Search path. The value must contain the exact ComponentSource path recorded in Wizard-Completion-Report.txt. Do not delete existing paths or the inherited $(DCC_UnitSearchPath) value.

> **Manual fallback screen reference.** RAD Studio path: Project > Options > Building > Delphi Compiler > Search path. Select All configurations and All platforms before editing. The Wizard normally performs this operation automatically; this reference exists for verification and recovery.

6. Build and Deploy the Language Packs

The JSON packs must be beside each executable under Localization\Languages. The Wizard configures a post-build event that uses Delphi's active DCC_ExeOutput value, so Debug, Release, Win32, Win64, and intentional custom output paths are handled by the build that produced the executable.

1. Build each target platform/configuration that will be tested or released.
2. Confirm each successful build reports language-pack deployment in its build output. The command runs PowerShell with -NoProfile -ExecutionPolicy Bypass.
3. For each executable, verify that Localization\Languages contains the English source pack and the translated target pack.
4. If an already-built output needs refreshing, select Redeploy Build Outputs. Copy Fallback Commands remains available for troubleshooting.
5. For a portable or USB copy, enter its full application folder on Step 3 so final processing and future builds deploy it automatically. Deploy New App Folder remains available for a new or temporary destination. The LanguagesFolder component property remains the relative value Localization\Languages, while the physical destination includes the drive.

> **PowerShell execution policy.** Generated commands launch C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe with -NoProfile -ExecutionPolicy Bypass. This bypass applies only to that launched process and prevents the deployment script from failing under the common local script-policy restriction.

7. Verify the Translated Application

1. Run the target executable from one deployed build folder.

2. Confirm that the language selector lists English and the translated language.
3. Choose the translated language. Open forms should update immediately without restarting the application.
4. Open secondary forms and pages. Confirm captions, labels, prompts, column headings, and other scanned static text use the selected language.
5. Confirm that live data, identifiers, dates, numeric values, selections, focused controls, and application connection state remain intact.
6. Close and restart the application. Confirm that the selected language is remembered.
7. Switch back to English and confirm that the English source pack restores the source text.
8. Repeat the test for every built Win32 and Win64 configuration intended for release.

> **Translation length is application-specific.** A correct translation can be substantially longer than its English source—20 to 50 percent growth is common for short interface phrases, and individual terms can grow more. The Review Center proposes conservative language-specific Width, Height, WordWrap, AutoSize, and grid-column rules and shows current and proposed geometry in its Visual Review. Accepted rules are stored in the JSON pack, applied only for that language, and reversed when another language is selected. Complex collisions, graph geometry, and substantial redesigns still require developer review in the target Form Designer.

8. Files Created by the Wizard

Location or file	Purpose
<Target>\Localization\Development\<Project>.<locale>.translation-project.json	Detailed working catalog retained for later rescans, updates, and review.
<Target>\Localization\Languages\<locale>.json	Compact runtime pack for deployment beside the application.
Documents\Delphi App Translation Backups\<Project>\<timestamp>.zip	Required pre-processing safety backup.
<Target>.dproj marked Wizard block	Inherits ComponentSource through all configurations/platforms and runs pack deployment after builds.
<Studio>\export\component-integration\<Project>	Generated component integration kit.

Location or file	Purpose
ComponentSource	Framework-neutral and VCL/FMX component/runtime units needed by the target.
Deploy-LanguagePacks.ps1	Copies the kit's Localization folder to a specified executable directory.
component-integration.json	Machine-readable integration manifest.
README.txt	Project-specific manual integration instructions.
Wizard-Completion-Report.txt	Final record of project, language, catalog, pack, backup, kit, manual step, and commands.

9. Update, Resume, or Add Another Language

9.1 Update an Existing Translation After UI or Text Changes

Changing an existing application is normal. New labels, revised captions, renamed controls, added forms, and changed resourcestrings must be saved and rescanned before they can enter the JSON catalog. The Wizard reuses the existing catalog and preserves matching work; it does not start the language over.

For the fewest provider calls and the clearest review, gather related interface changes into one practical batch. A single urgent correction can still be processed by itself.

1. Make the intended UI or source-text changes in Delphi, then choose File > Save All. The scanner reads saved DFM/FMX and Pascal files, not unsaved Form Designer buffers.
2. Run the current Translation Studio and start the Setup Wizard. Select the same .dproj, source language, target language, and provider used for the existing catalog.
3. Run Scan Project again. Read the new, changed, unchanged, and obsolete counts. A renamed component receives a new stable key; its former key remains Obsolete for traceability.
4. Continue to final processing. Automatic translation sends only eligible new, changed, or otherwise unresolved entries. Matching Reviewed and Approved translations are preserved.
5. If any target DFM/FMX or Pascal file is saved after the scan, final processing stops instead of exporting stale results. Return to Scan Project and scan again.

6. The already installed design package, manager, selector, and configured search path normally remain in place. Review the Delphi Component step and acknowledge the existing setup; do not add duplicate components.
7. Let final processing validate the catalog, export fresh JSON, regenerate the kit, and deploy to known build outputs and every available configured application destination.
8. Rebuild the target because its UI/source changed. Test every affected form in the source and target languages, including text wrapping, alignment, control state, and language persistence.

> **Repeatable update sequence.** Batch changes -> Save All -> Scan -> Translate unresolved -> Validate -> Export -> Deploy -> Rebuild -> Test.

9.2 What the Wizard Repeats Automatically

- Completed steps may be revisited from the left rail while the Wizard remains open and final processing is not running.
- Selecting a different project clears downstream scan/catalog state. Run the scan again.
- Selecting a different target language creates or opens that language's own development catalog and also requires a new scan in the Wizard session.
- A later scan preserves matching translations, marks changed source text for attention, adds new entries, and retains removed entries as obsolete.
- Automatic translation sends only eligible unresolved entries. Existing reviewed and approved work is preserved.
- Run the Wizard again when adding another language. Deploy all generated packs to each executable folder and refresh or restart the target application as required by its integration.
- The Wizard can rescan, merge, translate unresolved entries, validate, export, regenerate the component kit, configure supported project metadata, and deploy packs. It cannot save an open Delphi editor buffer, decide whether a translation is linguistically ideal, or redesign controls for longer text.

10. Troubleshooting

Problem	What it means	Action
Project browse or identification fails	An old Wizard build may be running, or the selected file lacks recognizable VCL/FMX metadata.	Use the current executable, select the .dproj rather than .dpr, and confirm the project opens normally in RAD Studio.

Problem	What it means	Action
Connection test fails	Provider authentication, billing, restriction, quota, plan, or network setup is incomplete.	Correct the provider account before scanning or final processing.
Scan returns zero	The wrong project was selected or the forms are not readable text resources.	Verify the project path and convert binary DFM resources through Delphi.
Unresolved count is lower than scan count	Some entries are dynamic, excluded, obsolete, protected, or already translated.	This is normally correct; review classifications in the full Studio when needed.
Package or DAT Localization is unavailable	The design package is not built, installed, or enabled in this RAD Studio installation.	Build the package set if needed, then use Component > Install Packages > Add with the exact Win32 Release design BPL.
Packs are absent after a build	The marked DPROJ settings are missing, the kit moved, the destination was unavailable, or the post-build command failed.	Inspect the build output and configured destination, verify the kit path, then use Redeploy Build Outputs, Deploy New App Folder, or the documented manual fallback.
Selector is empty	Packs are missing, invalid, or have a different applicationId.	Check Localization\Languages beside the running executable and verify ApplicationId.
Some text remains English	The text may be dynamic, excluded, added after the scan, or not wired as a resourcestring.	Rescan and translate new entries; use the full Studio to inspect runtime classifications and manual wiring warnings.
A newly added label is missing from the scan	The form was not saved, the wrong project/catalog is open, or the new property is not an eligible text property.	Choose File > Save All in Delphi, confirm the same .dproj and target language, then rescan and inspect the entry classification.

Problem	What it means	Action
Final processing says source was saved after the scan	The scan snapshot is stale and exporting it could omit a recent UI or source-text change.	Choose Save All in Delphi, return to Scan Project, scan again, and then continue. Do not bypass this safeguard.
Translated text clips or overlaps	The target layout was sized only for the shorter source language.	In the Delphi Form Designer, enable appropriate wrapping/AutoSize and adjust layout containers, margins, or control dimensions; then rebuild and visually retest every supported language.

11. Where to Learn More

- User Guide: full manual workflow, provider setup, catalog review, validation, export, integration, troubleshooting, security, and folder reference.
- Engineering Guide: architecture, JSON schema, scanners, provider clients, runtime managers, packages, lifecycle, security, and validation strategy.
- Generated kit README.txt: exact instructions for the selected target project and framework.
- Wizard-Completion-Report.txt: exact paths and commands produced by the completed Wizard run.